

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 4

Технический дизайн микросервисной архитектуры

Выполнил:
Мальцев И. И.
БР 1.2

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Разделить монолитное backend-приложение сайта поиска работы на набор микросервисов. Для каждого сервиса нужно определить зону ответственности, отдельную базу данных, способы взаимодействия с другими сервисами и основные эндпоинты.

В качестве исходной системы используется REST API для сайта поиска работы: авторизация, профили соискателей и работодателей, резюме, вакансии, навыки, отрасли и отклики.

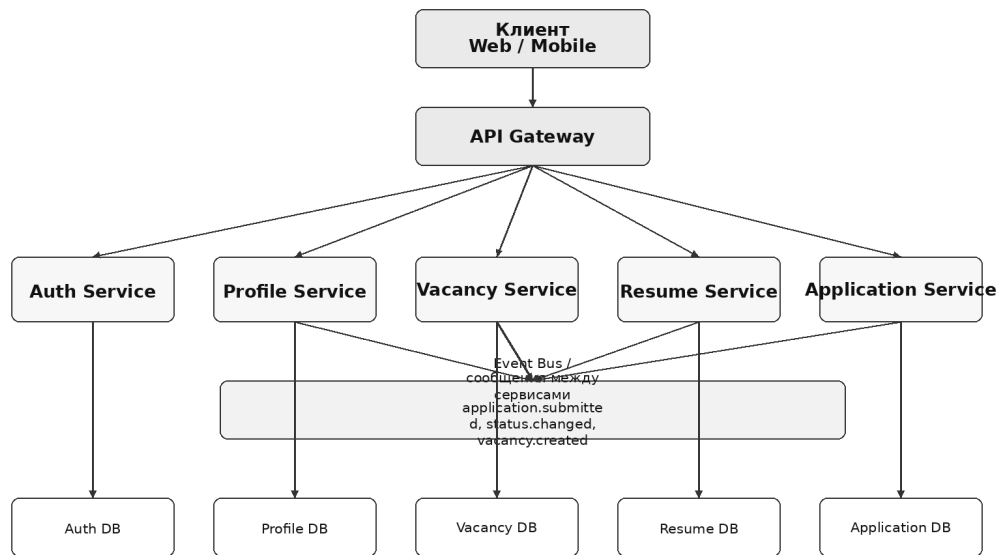
Ход работы

Сначала были выделены основные части монолита. Логика приложения хорошо делится по бизнес-областям: авторизация, профили, вакансии, резюме и отклики. Поэтому для микросервисной версии была выбрана схема database-per-service, где каждый сервис владеет только своей базой данных.

Состав микросервисов:

- API Gateway - единая точка входа для внешних клиентов. Он принимает запросы, проверяет токен и перенаправляет запрос в нужный сервис.
- Auth Service - регистрация, вход, проверка JWT-токена, хранение пользователей и refresh-токенов.
- Profile Service - данные соискателей, работодателей и компаний.
- Vacancy Service - вакансии, отрасли, навыки и требования к вакансии.
- Resume Service - резюме соискателей и навыки, указанные в резюме.
- Application Service - отклики на вакансии и изменение статусов откликов работодателем.

Общая схема взаимодействия сервисов представлена ниже.



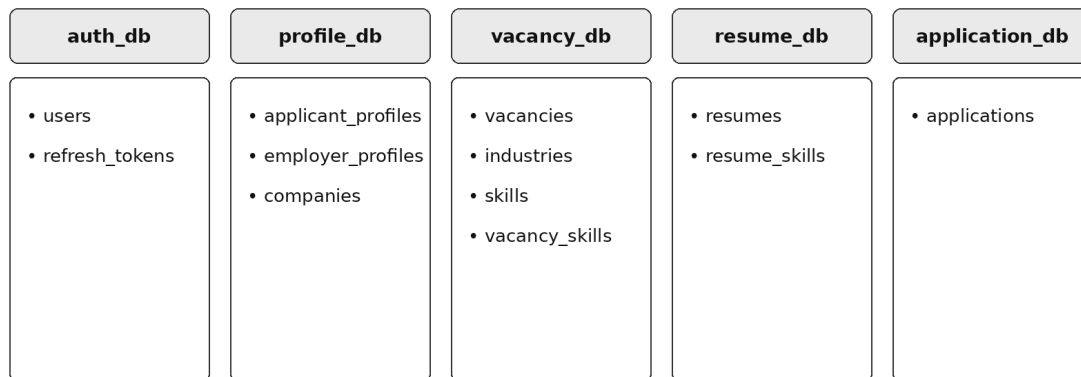
Каждый сервис работает со своей базой данных. Прямой доступа к чужой БД нет, обмен идет через API или события.

Рисунок 1 - Микросервисная архитектура приложения

Основной принцип: сервисы не обращаются напрямую к чужим базам данных. Если Application Service нужно проверить вакансию или резюме, он обращается к Vacancy Service и Resume Service через внутренние HTTP-эндпоинты. Для событий, которые не требуют мгновенного ответа, можно использовать сообщения: например, application.submitted или application.status_changed.

Разделение баз данных

База данных монолитного приложения была разделена по зонам ответственности сервисов. В таблицах между сервисами сохраняются только идентификаторы связанных сущностей. Например, отклик хранит vacancy_id и resume_id, но не хранит полные данные вакансии и резюме.



Связи между сервисами хранятся по идентификаторам: user_id, company_id, vacancy_id, resume_id.

Application Service не лезет напрямую в БД вакансий или резюме: он запрашивает данные через внутренние API.

Такое разделение соответствует database-per-service и позволяет развивать сервисы отдельно.

Рисунок 2 - Разделение базы данных по сервисам

Сервис	База данных	Основные таблицы
Auth Service	auth_db	users, refresh_tokens
Profile Service	profile_db	applicant_profiles, employer_profiles, companies
Vacancy Service	vacancy_db	vacancies, industries, skills, vacancy_skills
Resume Service	resume_db	resumes, resume_skills
Application Service	application_db	applications

Взаимодействие сервисов

Для внешних запросов используется API Gateway. Клиент не знает, в какой сервис пойдет запрос: он работает только с публичным REST API. Внутри системы сервисы общаются двумя способами: синхронно через HTTP и асинхронно через события.

- Синхронный HTTP используется там, где нужен быстрый ответ: проверка токена, получение вакансии, получение резюме, проверка профиля работодателя.
- Асинхронные события подходят для уведомлений и фоновой обработки: создан отклик, изменился статус отклика, опубликована вакансия.

Пример сценария отклика на вакансию:

- 1. Клиент отправляет POST /api/v1/vacancies/{id}/applications через API Gateway.

- 2. Gateway проверяет JWT через Auth Service и передает запрос в Application Service.
- 3. Application Service запрашивает Vacancy Service, чтобы убедиться, что вакансия существует и опубликована.
- 4. Application Service запрашивает Resume Service, чтобы проверить резюме пользователя.
- 5. После проверки создается отклик со статусом submitted.
- 6. Сервис публикует событие application.submitted для дальнейшей обработки.

Основные эндпоинты

Сервис	Эндпоинт	Назначение
Auth	POST /internal/auth/validate	Проверка JWT-токена для внутренних сервисов
Profile	GET /internal/applicants/{id}	Получение профиля соискателя
Profile	GET /internal/employers/{id}	Получение профиля работодателя
Vacancy	GET /internal/vacancies/{id}	Проверка и получение вакансии
Resume	GET /internal/resumes/{id}	Проверка и получение резюме
Application	POST /api/v1/vacancies/{id}/applications	Создание отклика
Application	PATCH /api/v1/employer/applications/{id}/status	Изменение статуса отклика

Пример тела запроса для создания отклика:

```
POST /api/v1/vacancies/{vacancyId}/applications
{
  "resume_id": "11111111-1111-4111-8111-111111111111",
  "cover_letter": "Здравствуйте, хочу откликнуться на вакансию."
}
```

Пример успешного ответа:

```
{
  "id": "22222222-2222-4222-8222-222222222222",
  "vacancy_id": "88888888-8888-4888-8888-888888888888",
  "resume_id": "11111111-1111-4111-8111-111111111111",
  "status": "submitted"
}
```

Ошибки

Код	Описание
400	Некорректное тело запроса или неверные параметры
401	Пользователь не авторизован или токен недействителен

403	Нет прав на выполнение действия
404	Вакансия, резюме или профиль не найдены
409	Конфликт данных, например повторный отклик на вакансию
500	Внутренняя ошибка сервиса

План разделения монолита

- Выделить общие модели и определить границы сервисов.
- Перенести авторизацию в Auth Service.
- Разделить таблицы по отдельным базам данных.
- Вынести вакансии, резюме и отклики в отдельные сервисы.
- Добавить внутренние API для проверки связанных сущностей.
- Подключить обмен событиями для откликов и уведомлений.
- Постепенно переключить внешний REST API на работу через API Gateway.

Вывод

В результате работы был подготовлен технический дизайн микросервисной архитектуры для backend-приложения сайта поиска работы. Монолит был разделен на несколько сервисов по бизнес-областям: авторизация, профили, вакансии, резюме и отклики.

Для каждого сервиса определена отдельная база данных, что соответствует подходу database-per-service. Также описаны основные способы взаимодействия между сервисами, внутренние эндпоинты, примеры запросов и возможные ошибки.

Полученная схема может использоваться как основа для дальнейшей реализации микросервисной версии приложения и постепенного перехода от монолитного REST API к распределенной архитектуре.